

MIS PRIMEROS

10 ALGORITMOS VISUALES

EN JAVASCRIPT — NIVEL 2

Arrays, Bucles y Objetos

Continúa desde donde dejaste el Nivel 1

 Nivel: Intermedio

 ~4-5 horas

 Arrays + Objetos

 Bucles for

Ya dominaste las bases: variables, condicionales, funciones y eventos onclick. Ahora das el siguiente paso. En este taller aprenderás a manejar colecciones de datos (arrays) y a representar entidades del mundo real (objetos), mientras sigues viendo resultados inmediatos en el navegador.

Regla de oro nivel 2: escribe cada línea a mano. Cuando un bucle for te confunda, dibuja en papel el valor de i en cada vuelta.

#	Ejercicio	Concepto nuevo	Dificultad
01	 Mi Primera Lista	Array + for básico	☆☆
02	 Agregar a la Lista	push() + re-render	☆☆
03	 Contador de Elementos	.length + estadísticas	☆☆
04	 Lista de Tareas con Objetos	Objetos { }	☆☆☆
05	 Marcar Tarea Completada	Índice + eventos dinámicos	☆☆☆
06	 Eliminar Elementos	splice()	☆☆☆
07	 Buscador en Tiempo Real	Filtrar + includes()	☆☆☆
08	 Galería con Objetos	Objetos complejos + grid	☆☆☆☆
09	 Calculadora de Promedio	Acumulador + estadísticas	☆☆☆☆
10	 Catálogo de Productos	CRUD completo	☆☆☆☆☆



Conceptos Nuevos en este Nivel — Referencia Rápida

Concepto	Sintaxis	En una oración
Array	<code>let a = [1, 2, 3]</code>	Lista ordenada de valores en corchetes
Acceso	<code>a[0] → 1</code>	Los índices empiezan en 0, no en 1
Bucle for	<code>for (let i=0; i<a.length; i++)</code>	Repite el bloque por cada elemento
.length	<code>a.length → 3</code>	Número de elementos en el array
push()	<code>a.push("nuevo")</code>	Agrega un elemento al final del array
splice()	<code>a.splice(1, 1)</code>	Elimina 1 elemento en la posición dada
includes()	<code>"hola".includes("ol")</code>	Devuelve true si el texto contiene la cadena
Objeto	<code>{ nombre:"Ana", edad:25 }</code>	Colección de pares clave:valor
Obj. propiedad	<code>persona.nombre → "Ana"</code>	Acceder a una propiedad con punto
Array de obj.	<code>[{...}, {...}, {...}]</code>	Array donde cada elemento es un objeto

01



Mi Primera Lista

Ejercicio 01 de 10 — Array básico + bucle for

Concepto

Crear un array de frutas y usar un bucle for para generar elementos en el DOM

Metáfora

Un array es como una lista del supermercado: varias cosas escritas en orden. El bucle for es el cajero que lee cada ítem uno por uno.

Código HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi Primera Lista</title>
  <style>
    body { font-family: sans-serif; max-width: 500px; margin: 60px auto; padding: 20px; }
    h2 { color: #1B6CA8; }
    ul { list-style: none; padding: 0; }
    li { background: #EEF6FD; border-left: 4px solid #1B6CA8; padding: 10px 16px; margin-bottom: 8px; border-radius: 0 8px 8px 0; font-size: 18px; }
    button { padding: 12px 24px; background: #1B6CA8; color: white; border: none; border-radius: 8px; cursor: pointer; font-size: 16px; margin-top: 16px; }
  </style>
</head>
<body>
  <h2>🍏 Mis Frutas Favoritas</h2>
  <button onclick="mostrarLista()">Mostrar lista</button>
  <ul id="lista-frutas"></ul>
</body>
</html>
```

Lógica en JavaScript (paso a paso)

```
// 1. Declaramos el array con nuestras frutas
//   Un array usa corchetes [] y separa elementos con comas
const frutas = ["🍏 Manzana", "🍌 Banana", "🍇 Uvas", "🍓 Fresa", "🥭 Mango"];

function mostrarLista() {
  // 2. Obtenemos el <ul> donde pondremos los ítems
  const lista = document.getElementById("lista-frutas");

  // 3. Limpiamos el contenido anterior (por si hacen clic varias veces)
```

```
lista.innerHTML = "";

// 4. El bucle for recorre el array desde el índice 0 hasta el último
//     frutas.length = 5 (número de elementos)
for (let i = 0; i < frutas.length; i++) {

    // 5. Creamos el HTML de un <li> con la fruta en posición i
    //     frutas[0] = "🍏 Manzana"
    //     frutas[1] = "🍌 Banana" ... y así hasta frutas[4]
    const item = "<li>" + frutas[i] + "</li>";

    // 6. += agrega el nuevo <li> SIN borrar los anteriores
    lista.innerHTML += item;
}
}
```

⚡ Mini-reto para ti

Cambia el array frutas para que tenga 8 elementos con tus favoritas.

Agrega un número de posición dentro de cada : "1. 🍏 Manzana", usando la variable i+1.

Bonus: Haz que el color de fondo de cada alterne entre azul claro y blanco.

Pista para el bonus: si $i \% 2 === 0$ el número es par, si no es impar.

02

+ Agregar Elementos a la Lista

Ejercicio 02 de 10 — push() + input + for para re-renderizar

Concepto

Usar push() para agregar datos al array y re-dibujar toda la lista con un bucle for

Metáfora

push() es como añadir una nota nueva al final de tu lista del supermercado. Después vuelves a leer la lista completa de arriba a abajo.

Código HTML

```
<body>
  <h2>+ Mi Lista Dinámica</h2>

  <div style="display: flex; gap: 8px; margin-bottom: 16px;">
    <input id="inp-item" type="text"
      placeholder="Escribe un ítem..."
      style="flex:1; padding:10px; font-size:16px;
        border:2px solid #ccc; border-radius:8px;">
    <button onclick="agregarItem()"
      style="padding:10px 20px; background:#2D6A4F;
        color:white; border:none; border-radius:8px;
        font-size:16px; cursor:pointer;">
      Agregar +
    </button>
  </div>

  <ul id="mi-lista"
    style="list-style:none; padding:0;"></ul>

  <style>
    body { font-family:sans-serif; max-width:500px; margin:60px auto; padding:20px; }
    li { background:#F0FBF1; border-left:4px solid #2D6A4F;
      padding:10px 16px; margin-bottom:8px;
      border-radius:0 8px 8px 0; font-size:17px; }
  </style>
</body>
```

Lógica en JavaScript (paso a paso)

```
// 1. Array global que empieza vacío
//    Lo declaramos fuera de la función para que persista
let items = [];

function agregarItem() {
  // 2. Leer el input
  const texto = document.getElementById("inp-item").value;
```

```

// 3. Validar: no agregar si está vacío
if (texto === "") {
  alert("Escribe algo primero");
  return;
}

// 4. push() agrega el nuevo texto AL FINAL del array
items.push(texto);

// 5. Limpiamos el input
document.getElementById("inp-item").value = "";

// 6. Re-dibujamos la lista completa
dibujarLista();
}

function dibujarLista() {
  const lista = document.getElementById("mi-lista");

  // 7. Borrar contenido anterior y re-construir
  lista.innerHTML = "";

  // 8. Bucle for para generar un <li> por cada ítem del array
  for (let i = 0; i < items.length; i++) {
    lista.innerHTML += "<li>" + items[i] + "</li>";
  }
}

```

⚡ Mini-reto para ti

Agrega un texto que muestre cuántos ítems hay en total: "Total: 3 ítems" usando `items.length`.

Haz que el array empiece con 3 valores por defecto: ["🥛 Leche", "🍞 Pan", "🥚 Huevos"].

Bonus: Permite también agregar presionando la tecla Enter. Pista: usa el evento `onkeydown` en el input y compara con `event.key === "Enter"`.

03

12
34

Contador de Elementos

Ejercicio 03 de 10 — .length + estadísticas del array

Concepto

Usar .length para mostrar cuántos elementos tiene el array, y actualizar ese contador en tiempo real

Metáfora

.length es como mirar el marcador de un juego: te dice exactamente cuántos elementos hay en ese momento. Si alguien anota, el marcador sube. Si se van, baja.

Código HTML

```
<body>
  <h2>📋 Gestión de Invitados</h2>

  <!-- Panel de estadísticas -->
  <div style="display:flex; gap:12px; margin-bottom:24px;">
    <div id="contador-total"
      style="flex:1; background:#C0392B; color:white; padding:20px;
        border-radius:12px; text-align:center;">
      <div style="font-size:48px; font-weight:bold">0</div>
      <div>Total invitados</div>
    </div>
    <div id="contador-max"
      style="flex:1; background:#922B21; color:white; padding:20px;
        border-radius:12px; text-align:center;">
      <div style="font-size:48px; font-weight:bold" id="quedan">10</div>
      <div>Lugares libres</div>
    </div>
  </div>

  <input id="inp-nombre" type="text" placeholder="Nombre del invitado"
    style="width:60%; padding:10px; font-size:16px;
      border:2px solid #ccc; border-radius:8px;">
  <button onclick="agregarInvitado()"
    style="padding:10px 18px; background:#C0392B; color:white;
      border:none; border-radius:8px; font-size:16px; cursor:pointer;">
    + Invitar
  </button>

  <ul id="lista-invitados" style="list-style:none; padding:0; margin-top:20px;"></ul>

  <style>
    body { font-family:sans-serif; max-width:560px; margin:40px auto; padding:20px; }
    li { background:#FFF5F5; border-left:4px solid #C0392B;
      padding:10px 16px; margin-bottom:8px; border-radius:0 8px 8px 0; }
  </style>
</body>
```



Lógica en JavaScript (paso a paso)

```
const LIMITE_MAXIMO = 10; // máximo de invitados permitidos
let invitados = [];

function agregarInvitado() {
  const nombre = document.getElementById("inp-nombre").value;

  if (nombre === "") {
    alert("Escribe un nombre");
    return;
  }

  // 1. Verificar si ya alcanzamos el límite usando .length
  if (invitados.length >= LIMITE_MAXIMO) {
    alert(";Lista llena! No hay más lugares.");
    return;
  }

  // 2. Agregar al array
  invitados.push(nombre);

  // 3. Limpiar input
  document.getElementById("inp-nombre").value = "";

  // 4. Actualizar todo
  actualizarContadores();
  dibujarLista();
}

function actualizarContadores() {
  // 5. .length nos da el número actual de elementos
  const total = invitados.length;
  const libres = LIMITE_MAXIMO - total;

  // 6. Actualizamos los paneles de estadísticas
  document.querySelector("#contador-total div").innerHTML = total;
  document.getElementById("quedan").innerHTML = libres;
}

function dibujarLista() {
  const lista = document.getElementById("lista-invitados");
  lista.innerHTML = "";

  for (let i = 0; i < invitados.length; i++) {
    // 7. Mostramos número de posición + nombre
    lista.innerHTML += "<li>" + (i+1) + ". " + invitados[i] + "</li>";
  }
}
```

⚡ Mini-reto para ti

Cambia el color del panel "Lugares libres" a rojo (#E74C3C) cuando queden 3 o menos.

Agrega un tercer panel que muestre el porcentaje de ocupación: $(\text{total} / \text{LIMITE_MAXIMO} * 100)$.toFixed(0) + "%".

Bonus: Si el array supera 8 elementos, muestra un aviso "⚠ ¡Casi lleno!" debajo del contador.

04



Lista de Tareas con Objetos

Ejercicio 04 de 10 — Objetos literales { } + array de objetos

Concepto

Representar cada tarea como un objeto { texto, completada } y usar esa propiedad para aplicar estilos visuales distintos

Metáfora

Un objeto es como una ficha de tarjeta de índice: tiene varios campos (nombre, estado, prioridad). Un array de objetos es un archivero lleno de fichas.

Código HTML

```
<body>
  <h2>✅ Mi Lista de Tareas</h2>

  <div style="display:flex; gap:8px; margin-bottom:20px;">
    <input id="inp-tarea" type="text" placeholder="Nueva tarea..."
      style="flex:1; padding:10px; font-size:16px;
        border:2px solid #ccc; border-radius:8px;">
    <button onclick="agregarTarea()"
      style="padding:10px 20px; background:#7B2FBE; color:white;
        border:none; border-radius:8px; font-size:16px; cursor:pointer;">
      Agregar
    </button>
  </div>

  <div id="lista-tareas"></div>

  <style>
    body { font-family:sans-serif; max-width:520px; margin:40px auto; padding:20px; }
    .tarea {
      display:flex; align-items:center; gap:12px;
      padding:12px 16px; margin-bottom:8px;
      border-radius:8px; border:1px solid #E9D8FF;
      background:#F5EEFF; font-size:17px; cursor:pointer;
      transition: opacity 0.3s;
    }
    .tarea.completada {
      background:#F3F4F6;
      text-decoration: line-through;
      color: #9CA3AF;
      opacity: 0.7;
    }
    .check { font-size:22px; }
  </style>
</body>
```



```

// 1. Array de objetos: cada tarea tiene "texto" y "completada"
let tareas = [
  { texto: "Aprender arrays en JS",    completada: false },
  { texto: "Practicar bucle for",      completada: false },
  { texto: "Completar el taller 1",    completada: true  },
];

function agregarTarea() {
  const texto = document.getElementById("inp-tarea").value;
  if (texto === "") return;

  // 2. Crear un nuevo objeto y agregarlo al array
  const nuevaTarea = {
    texto:      texto,
    completada: false    // siempre empieza pendiente
  };

  tareas.push(nuevaTarea);
  document.getElementById("inp-tarea").value = "";
  dibujarTareas();
}

function toggleTarea(indice) {
  // 3. Accedemos al objeto en la posición [indice]
  //    y cambiamos su propiedad completada
  tareas[indice].completada = !tareas[indice].completada;
  dibujarTareas();
}

function dibujarTareas() {
  const contenedor = document.getElementById("lista-tareas");
  contenedor.innerHTML = "";

  for (let i = 0; i < tareas.length; i++) {
    const tarea = tareas[i]; // accedemos al objeto en posición i

    // 4. El emoji del check cambia según el estado
    const checkEmoji = tarea.completada ? "✅" : "❑";

    // 5. La clase CSS cambia según el estado
    const claseCSS = tarea.completada ? "tarea completada" : "tarea";

    // 6. onclick llama a toggleTarea con el índice i
    contenedor.innerHTML +=
      "<div class=\"" + claseCSS + "\" onclick=\"toggleTarea(" + i + ")\">" +
      "  <span class=\"check\">" + checkEmoji + "</span>" +
      "  " + tarea.texto +
      "</div>";
  }
}

// Dibujamos al cargar la página
dibujarTareas();

```

⚡ Mini-reto para ti

Agrega un contador de "X completadas de Y" arriba de la lista.

Tip: recorre el array con un for y cuenta cuántas tienen completada === true.

Agrega una propiedad "prioridad" al objeto: "alta", "media", "baja". Muestra un emoji diferente (🔴🟡🟢) según la prioridad.

Bonus: Agrega un botón "Limpiar completadas" que elimine todas las tareas con completada === true.

05



Marcar Tarea como Completada

Ejercicio 05 de 10 — Eventos en elementos generados dinámicamente + índices

Concepto

Entender por qué el índice `i` es crítico para identificar qué objeto modificar dentro del array cuando el usuario hace clic

Metáfora

Imagina una fila de casilleros numerados. Para abrir el correcto necesitas su número (índice). Sin el número, no sabes cuál abrir.

Código HTML

```
<body>
  <h2>&img alt="Icon of a task or checklist item." data-bbox="161 358 178 371"/> Tablero de Tareas</h2>

  <div style="display:flex; gap:12px; margin-bottom:16px;">
    <input id="inp-nueva" type="text" placeholder="Tarea nueva..."
      style="flex:1; padding:10px; font-size:16px;
        border:2px solid #ccc; border-radius:8px;">
    <button onclick="agregar()"
      style="padding:10px 18px; background:#D68910; color:white;
        border:none; border-radius:8px; cursor:pointer; font-size:16px;">
      + Agregar
    </button>
  </div>

  <p id="stats" style="color:#6B7280; font-size:15px;"></p>
  <div id="tablero"></div>

  <style>
    body { font-family:sans-serif; max-width:520px; margin:40px auto; padding:20px; }
    .item {
      display:flex; align-items:center; justify-content:space-between;
      padding:14px 16px; margin-bottom:10px;
      border-radius:10px; background:#FCF3CF;
      border:1px solid #F9E79F; font-size:17px;
      cursor:pointer; user-select:none;
      transition: all 0.25s ease;
    }
    .item:hover { transform: translateX(4px); }
    .item.hecho { background:#D5F5E3; border-color:#A9D0D9;
      text-decoration:line-through; color:#7D8B7D; }
    .badge { font-size:11px; background:white; padding:2px 8px;
      border-radius:999px; color:#888; border:1px solid #ddd; }
  </style>
</body>
```



```

let lista = [
  { texto:"Revisar el correo",      hecho:false },
  { texto:"Llamar al cliente",     hecho:false },
  { texto:"Actualizar portafolio",  hecho:true  },
];

function agregar() {
  const txt = document.getElementById("inp-nueva").value;
  if (!txt) return;
  lista.push({ texto: txt, hecho: false });
  document.getElementById("inp-nueva").value = "";
  render();
}

// Esta función recibe el índice del elemento que se tocó
function toggle(id) {
  // Invertimos el valor booleano de la propiedad "hecho"
  lista[id].hecho = !lista[id].hecho;
  render(); // re-dibujamos todo
}

function render() {
  const tablero = document.getElementById("tablero");
  tablero.innerHTML = "";

  // Contador de completadas
  let completadas = 0;
  for (let i = 0; i < lista.length; i++) {
    if (lista[i].hecho) completadas++;
  }

  document.getElementById("stats").innerHTML =
    completadas + " completadas de " + lista.length;

  // Generar cada elemento
  for (let i = 0; i < lista.length; i++) {
    const item = lista[i];
    const clase = item.hecho ? "item hecho" : "item";
    const icon = item.hecho ? "✅" : "❌";

    // CLAVE: pasamos i (índice) al onclick
    // Cuando el usuario haga clic, toggle sabrá QUÉ ítem cambiar
    tablero.innerHTML +=
      "<div class=\"" + clase + "\" onclick=\"toggle(" + i + ")\">\" +
      "  <span>\" + icon + "  \" + item.texto + "</span>\" +
      "  <span class=\"badge\">#\" + i + "</span>\" +
      "</div>";
  }
}

render(); // Primera llamada al cargar

```

⚡ Mini-reto para ti

Agrega una animación de celebración: cuando `completadas === lista.length`, muestra "🎉 ¡Todo listo!" en el stats.

Cambia el badge para mostrar la hora en que se completó. Pista: guarda `new Date().toLocaleTimeString()` en el objeto cuando se marca como hecho.

Bonus: Ordena el array para que las tareas pendientes aparezcan primero y las completadas al final, sin cambiar los índices originales.

06



Eliminar Elementos del Array

Ejercicio 06 de 10 — splice() para eliminar por índice

Concepto

Usar splice(indice, 1) para eliminar un elemento del array por su posición y re-renderizar la lista

Metáfora

splice() es como arrancar una hoja específica de una libreta. Dices "arranca la hoja número 3" y la libreta se reordena sola.



Código HTML

```
<body>
  <h2>&img alt="Trash can icon" data-bbox="161 358 178 371"/> Lista con Eliminación</h2>

  <div style="display:flex; gap:8px; margin-bottom:20px;">
    <input id="inp-elem" type="text" placeholder="Agregar elemento..."
      style="flex:1; padding:10px; font-size:16px;
        border:2px solid #ccc; border-radius:8px;">
    <button onclick="agregar()"
      style="padding:10px 20px; background:#148F77; color:white;
        border:none; border-radius:8px; cursor:pointer; font-size:16px;">
      + Agregar
    </button>
  </div>

  <div id="contenedor"></div>

  <style>
    body { font-family:sans-serif; max-width:520px; margin:40px auto; padding:20px; }
    .fila {
      display:flex; align-items:center; justify-content:space-between;
      padding:12px 16px; margin-bottom:8px;
      background:#D1F2EB; border-left:4px solid #148F77;
      border-radius:0 8px 8px 0; font-size:17px;
    }
    .btn-eliminar {
      background:#E74C3C; color:white; border:none;
      padding:6px 12px; border-radius:6px; cursor:pointer;
      font-size:14px; font-weight:bold;
    }
    .btn-eliminar:hover { background:#C0392B; }
  </style>
</body>
```



Lógica en JavaScript (paso a paso)

```

let elementos = ["🥑 Aguacate", "🍋 Limón", "🫐 Arándanos", "🍑 Durazno"];

function agregar() {
  const texto = document.getElementById("inp-elem").value;
  if (!texto) return;
  elementos.push(texto);
  document.getElementById("inp-elem").value = "";
  render();
}

function eliminar(indice) {
  // splice(posición, cantidadAEliminar)
  // splice(2, 1) elimina 1 elemento empezando en la posición 2
  elementos.splice(indice, 1);

  // Re-dibujamos después de eliminar
  render();
}

function render() {
  const contenedor = document.getElementById("contenedor");
  contenedor.innerHTML = "";

  // Si el array está vacío, mostramos un mensaje
  if (elementos.length === 0) {
    contenedor.innerHTML = "<p style='color:#888;text-align:center;'>Lista vacía 🤖</p>";
    return;
  }

  for (let i = 0; i < elementos.length; i++) {
    // Cada fila tiene el texto y un botón que llama a eliminar(i)
    contenedor.innerHTML +=
      "<div class='fila'>" +
      "  <span>" + elementos[i] + "</span>" +
      "  <button class='btn-eliminar' onclick='eliminar(" + i + ")'>" +
      "    🗑 Eliminar</button>" +
      "</div>";
  }
}

render();

```

⚡ Mini-reto para ti

Agrega un botón "🗑 Limpiar todo" que vacíe el array completo: `elementos = []`; `render()`;

Agrega confirmación antes de eliminar: `if (confirm("¿Eliminar este elemento?")) { splice... }`

Bonus: Implementa un botón "Deshacer" que recupere el último elemento eliminado. Pista: guarda el elemento en una variable antes de hacer `splice`.

07



Buscador Simple en Tiempo Real

Ejercicio 07 de 10 — Filtrar array con bucle for + oninput

Concepto

Usar el evento oninput para ejecutar código cada vez que el usuario escribe, y un bucle for para mostrar solo los elementos que coinciden con la búsqueda

Metáfora

Es como buscar en un directorio telefónico: a medida que escribes letras, los resultados se van filtrando hasta mostrar solo los que coinciden.

Código HTML

```
<body>
  <h2>&img alt="Magnifying glass icon" Buscador de Países</h2>

  <input
    id="inp-buscar"
    type="text"
    placeholder="Escribe para buscar..."
    oninput="buscar()"
    style="width:100%; padding:14px; font-size:18px;
      border:2px solid #A569BD; border-radius:10px;
      box-sizing:border-box; margin-bottom:8px;">

  <p id="resultado-conteo" style="color:#6B7280; font-size:14px;"></p>
  <ul id="resultado-lista" style="list-style:none; padding:0;"></ul>

  <style>
    body { font-family:sans-serif; max-width:520px; margin:40px auto; padding:20px; }
    li { padding:12px 16px; margin-bottom:6px;
      background:#E8DAEF; border-radius:8px; font-size:17px;
      border-left:4px solid #A569BD; }
    .highlight { background:#F5EEF8; }
  </style>
</body>
```



Lógica en JavaScript (paso a paso)

```
// Dataset: array de países
const paises = [
  "🇦🇷 Argentina", "🇧🇷 Brasil", "🇨🇱 Chile",
  "🇨🇴 Colombia", "🇪🇨 Ecuador", "🇲🇽 México",
  "🇵🇪 Perú", "🇺🇾 Uruguay", "🇻🇪 Venezuela",
  "🇵🇦 Panamá", "🇨🇷 Costa Rica", "🇧🇴 Bolivia",
];
```

```

function buscar() {
  // 1. Leer lo que el usuario está escribiendo
  //     toLowerCase() convierte a minúsculas para comparar sin importar mayúsculas
  const termino = document.getElementById("inp-buscar").value.toLowerCase();

  // 2. Array vacío donde guardaremos los que coincidan
  let encontrados = [];

  // 3. Recorrer el array original buscando coincidencias
  for (let i = 0; i < paises.length; i++) {
    const pais = paises[i].toLowerCase();

    // includes() devuelve true si el texto contiene el término buscado
    if (pais.includes(termino)) {
      encontrados.push(paises[i]); // guardamos el original (con emoji)
    }
  }

  // 4. Actualizar contador
  document.getElementById("resultado-conteo").innerHTML =
    encontrados.length + " de " + paises.length + " países";

  // 5. Mostrar resultados
  const lista = document.getElementById("resultado-lista");
  lista.innerHTML = "";

  if (encontrados.length === 0) {
    lista.innerHTML = "<li style=\"background:#FEF2F2; color:#DC2626;\">
      Sin resultados para: " + termino + "</li>";
    return;
  }

  for (let i = 0; i < encontrados.length; i++) {
    lista.innerHTML += "<li>" + encontrados[i] + "</li>";
  }
}

// Mostrar todos al cargar
buscar();

```

⚡ Mini-reto para ti

Cambia el array por tu lista de 15 películas, canciones o videojuegos favoritos.

Agrega un botón "Limpiar búsqueda" que vacíe el input y vuelva a mostrar todos.

Bonus avanzado: resalta el texto buscado dentro del resultado. Pista: usa `pais.replace(termino, "<strong>" + termino + "</strong>")` antes de insertar en el HTML.

08



Galería de Imágenes con Objetos

Ejercicio 08 de 10 — Array de objetos + generación de tarjetas visuales

Concepto

Crear un array donde cada objeto tiene múltiples propiedades (nombre, url, categoría) y generar tarjetas HTML complejas con un solo bucle for

Metáfora

Es como una base de datos de fotos de tu teléfono: cada foto tiene título, fecha, álbum. El bucle es el visor que muestra todas las fotos una por una.

Código HTML

```
<body>
  <h2><img alt="Landscape icon" data-bbox="161 358 176 373"/> Galería de Paisajes</h2>

  <div id="galeria"
    style="display:grid; grid-template-columns:repeat(auto-fill, minmax(200px,1fr));
    gap:16px;">
  </div>

  <div id="modal-info"
    style="display:none; margin-top:24px; padding:20px;
    background:#EBF5FB; border-radius:12px;
    border-left:4px solid #2E86C1;">
  </div>

  <style>
    body { font-family:sans-serif; max-width:700px; margin:40px auto; padding:20px; }
    .tarjeta {
      border-radius:12px; overflow:hidden;
      box-shadow:0 2px 12px rgba(0,0,0,0.12);
      cursor:pointer; transition:transform 0.2s;
      background:white;
    }
    .tarjeta:hover { transform:translateY(-6px); }
    .tarjeta img { width:100%; height:150px; object-fit:cover; display:block; }
    .tarjeta-info { padding:12px; }
    .tarjeta-nombre { font-weight:bold; font-size:15px; color:#1A5276; }
    .tarjeta-cat { font-size:12px; color:#888; margin-top:4px; }
    .badge-cat {
      display:inline-block; background:#D6EAF8; color:#1A5276;
      padding:2px 8px; border-radius:999px; font-size:11px;
    }
  </style>
</body>
```



Lógica en JavaScript (paso a paso)

```
// Array de objetos: cada uno representa una imagen con metadatos
const fotos = [
  {
    nombre: "Montaña Nevada",
    url: "https://picsum.photos/seed/mountain/400/300",
    categoria: "Naturaleza",
    descripcion: "Picos cubiertos de nieve al amanecer."
  },
  {
    nombre: "Playa Tropical",
    url: "https://picsum.photos/seed/beach/400/300",
    categoria: "Mar",
    descripcion: "Aguas cristalinas y arena blanca."
  },
  {
    nombre: "Bosque Profundo",
    url: "https://picsum.photos/seed/forest/400/300",
    categoria: "Naturaleza",
    descripcion: "Árboles centenarios y neblina suave."
  },
  {
    nombre: "Ciudad de Noche",
    url: "https://picsum.photos/seed/city/400/300",
    categoria: "Urbano",
    descripcion: "Rascacielos iluminados al anochecer."
  },
  {
    nombre: "Desierto Rojo",
    url: "https://picsum.photos/seed/desert/400/300",
    categoria: "Naturaleza",
    descripcion: "Dunas de arena bajo un cielo naranja."
  },
  {
    nombre: "Lago de Montaña",
    url: "https://picsum.photos/seed/lake/400/300",
    categoria: "Agua",
    descripcion: "Reflejo perfecto de los picos en el agua."
  },
];

function renderGaleria() {
  const galeria = document.getElementById("galeria");
  galeria.innerHTML = "";

  for (let i = 0; i < fotos.length; i++) {
    const foto = fotos[i]; // accedemos al objeto en posición i

    // Construimos el HTML de cada tarjeta usando las propiedades del objeto
    galeria.innerHTML +=
      "<div class=\"tarjeta\" onclick=\"verDetalle(" + i + ")\">" +
      "  <img src=\"" + foto.url + "\" alt=\"" + foto.nombre + "\">" +
      "  <div class=\"tarjeta-info\">" +
      "    <div class=\"tarjeta-nombre\">" + foto.nombre + "</div>" +
      "    <span class=\"badge-cat\">" + foto.categoria + "</span>" +
      "  </div>" +
      "</div>";
  }
}
```

```
}

function verDetalle(i) {
  const foto = fotos[i];
  const modal = document.getElementById("modal-info");
  modal.style.display = "block";
  modal.innerHTML =
    "<strong>👤 " + foto.nombre + "</strong><br>" +
    "<em>Categoría: " + foto.categoria + "</em><br>" +
    "<p>" + foto.descripcion + "</p>";
}

renderGaleria();
```

⚡ Mini-reto para ti

Agrega una propiedad "likes: 0" a cada objeto y un botón ❤️ en cada tarjeta que incremente el likes de ese objeto y lo muestre.

Agrega un selector <select> con las categorías y filtra la galería para mostrar solo fotos de esa categoría.

Bonus: Agrega una propiedad "destacada: true/false" y muestra las fotos destacadas con un borde dorado especial.

09



Calculadora de Promedio con Notas

Ejercicio 09 de 10 — Array de números + acumulador + cálculo estadístico

 Concepto

Usar un bucle for con una variable acumuladora para sumar todos los elementos y dividir entre `.length` para obtener el promedio

 Metáfora

Es como calcular el promedio de temperatura de la semana: sumas todos los días y divides entre 7. El bucle hace la suma automáticamente sin importar cuántos días haya.

 Código HTML

```
<body>
  <h2> Calculadora de Notas</h2>

  <div style="display:flex; gap:8px; margin-bottom:20px;">
    <input id="inp-nota" type="number" min="0" max="10"
      placeholder="Nota (0-10)"
      style="flex:1; padding:10px; font-size:16px;
        border:2px solid #ccc; border-radius:8px;">
    <button onclick="agregarNota()"
      style="padding:10px 18px; background:#E74C3C;
        color:white; border:none; border-radius:8px;
        font-size:16px; cursor:pointer;">
      Agregar
    </button>
  </div>

  <!-- Panel de estadísticas -->
  <div style="display:grid; grid-template-columns:repeat(4,1fr); gap:10px; margin-
bottom:20px;">
    <div style="background:#E74C3C;color:white;padding:16px;border-radius:10px;text-
align:center">
      <div id="stat-prom" style="font-size:32px;font-weight:bold"></div>
      <div style="font-size:12px">Promedio</div>
    </div>
    <div style="background:#8E44AD;color:white;padding:16px;border-radius:10px;text-
align:center">
      <div id="stat-max" style="font-size:32px;font-weight:bold"></div>
      <div style="font-size:12px">Máxima</div>
    </div>
    <div style="background:#2E86C1;color:white;padding:16px;border-radius:10px;text-
align:center">
      <div id="stat-min" style="font-size:32px;font-weight:bold"></div>
      <div style="font-size:12px">Mínima</div>
    </div>
    <div style="background:#1ABC9C;color:white;padding:16px;border-radius:10px;text-
align:center">
      <div id="stat-total" style="font-size:32px;font-weight:bold">0</div>
      <div style="font-size:12px">Total notas</div>
    </div>
  </div>
```

```

</div>

<div id="barras"></div>

<style>
  body { font-family:sans-serif; max-width:560px; margin:40px auto; padding:20px; }
  .barra-fila { display:flex; align-items:center; gap:10px; margin-bottom:6px; }
  .barra { height:24px; background:#E74C3C; border-radius:4px; transition:width 0.4s; }
</style>
</body>

```



Lógica en JavaScript (paso a paso)

```

let notas = [];

function agregarNota() {
  const valor = Number(document.getElementById("inp-nota").value);

  // Validar rango
  if (isNaN(valor) || valor < 0 || valor > 10) {
    alert("Ingresa una nota entre 0 y 10");
    return;
  }

  notas.push(valor);
  document.getElementById("inp-nota").value = "";
  calcularEstadisticas();
  dibujarBarras();
}

function calcularEstadisticas() {
  if (notas.length === 0) return;

  // 1. PROMEDIO: acumulador + división
  let suma = 0;
  for (let i = 0; i < notas.length; i++) {
    suma = suma + notas[i]; // sumamos cada nota
  }
  const promedio = suma / notas.length;

  // 2. MÁXIMA: empezamos con la primera nota y comparamos
  let maximo = notas[0];
  for (let i = 1; i < notas.length; i++) {
    if (notas[i] > maximo) {
      maximo = notas[i];
    }
  }

  // 3. MÍNIMA: igual pero buscamos el menor
  let minimo = notas[0];

```

```

for (let i = 1; i < notas.length; i++) {
  if (notas[i] < minimo) {
    minimo = notas[i];
  }
}

// 4. Actualizar la interfaz
document.getElementById("stat-prom").innerHTML = promedio.toFixed(1);
document.getElementById("stat-max").innerHTML = maximo;
document.getElementById("stat-min").innerHTML = minimo;
document.getElementById("stat-total").innerHTML = notas.length;
}

function dibujarBarras() {
  const contenedor = document.getElementById("barras");
  contenedor.innerHTML = "";

  for (let i = 0; i < notas.length; i++) {
    // El ancho de la barra es proporcional a la nota (máximo 10)
    const porcentaje = (notas[i] / 10) * 100;
    const color = notas[i] >= 6 ? "#27AE60" : "#E74C3C";

    contenedor.innerHTML +=
      "<div class=\"barra-fila\">" +
      "  <span style=\"width:20px; text-align:right;\">" + (i+1) + "</span>" +
      "  <div class=\"barra\" "
        style=\"width:\" + porcentaje + \"%; background:\" + color + "\"></div>" +
      "  <span style=\"font-size:14px;\">" + notas[i] + "</span>" +
      "</div>";
  }
}

```

⚡ Mini-reto para ti

Agrega un indicador de "estado" según el promedio: $< 6.0 \rightarrow$ **✗ Reprobado** en rojo; $\geq 6.0 \rightarrow$ **✅ Aprobado** en verde; $\geq 9.0 \rightarrow$ **🏆 Excelente** en dorado.

Calcula cuántas notas son aprobatorias (≥ 6) y cuántas son reprobatorias. Muéstralo como fracción.

Bonus: Agrega un botón "Eliminar última nota" que use `notas.pop()` (elimina el último elemento del array).

10



Generador de Catálogo de Productos

Ejercicio 10 de 10 — Array de objetos complejos + CRUD visual completo

Concepto

Combinar todo lo aprendido: objetos con múltiples propiedades, bucle for para renderizar tarjetas ricas, agregar, filtrar y calcular totales

Metáfora

Es un mini e-commerce: el array es el inventario de la bodega, el bucle es el empleado que coloca cada producto en la vitrina, y las funciones son las acciones de la caja registradora.

Código HTML

```
<body>
  <h2>🛍 Mi Catálogo de Productos</h2>

  <!-- Formulario de nuevo producto -->
  <div style="background:#EAF7F7; padding:16px; border-radius:12px;
    margin-bottom:20px; border:1px solid #A3E4D7;">
    <strong style="color:#148F77;">+ Agregar producto</strong>
    <div style="display:grid; grid-template-columns:1fr 1fr; gap:8px; margin-
top:10px;">
      <input id="inp-nombre" type="text" placeholder="Nombre"
        style="padding:8px; border:1px solid #ccc; border-radius:6px; font-
size:15px;">
      <input id="inp-precio" type="number" placeholder="Precio" min="0"
        style="padding:8px; border:1px solid #ccc; border-radius:6px; font-
size:15px;">
      <input id="inp-cat" type="text" placeholder="Categoría"
        style="padding:8px; border:1px solid #ccc; border-radius:6px; font-
size:15px;">
      <input id="inp-emoji" type="text" placeholder="Emoji (ej: 🍷)"
        style="padding:8px; border:1px solid #ccc; border-radius:6px; font-
size:15px;">
    </div>
    <button onclick="agregarProducto()"
      style="width:100%; margin-top:10px; padding:10px; background:#1ABC9C;
        color:white; border:none; border-radius:8px;
        font-size:16px; cursor:pointer;">
      Agregar al catálogo +
    </button>
  </div>

  <!-- Barra de totales -->
  <div id="resumen"
    style="display:flex; gap:12px; margin-bottom:16px; font-size:15px;"></div>

  <!-- Grid de tarjetas -->
  <div id="catalogo"
    style="display:grid; grid-template-columns:repeat(auto-fill,minmax(180px,1fr));
    gap:14px;">
```

```

</div>

<style>
  body { font-family:sans-serif; max-width:660px; margin:30px auto; padding:20px; }
  .producto-card {
    background:white; border:1px solid #D1F2EB;
    border-radius:12px; padding:16px; text-align:center;
    box-shadow:0 2px 8px rgba(0,0,0,0.08);
    transition:transform 0.2s;
  }
  .producto-card:hover { transform:translateY(-4px); }
  .prod-emoji { font-size:48px; }
  .prod-nombre { font-weight:bold; margin:8px 0 4px; color:#0E6655; }
  .prod-precio { color:#1ABC9C; font-size:18px; font-weight:bold; }
  .prod-cat { font-size:11px; color:#888;
    background:#EAF7F7; padding:2px 8px;
    border-radius:999px; display:inline-block;
    margin-top:6px; }
  .btn-del { margin-top:10px; padding:4px 10px;
    background:#FDEDEC; color:#E74C3C;
    border:1px solid #FADADD; border-radius:6px;
    cursor:pointer; font-size:13px; }
  .stat-box { flex:1; background:#EAF7F7; padding:12px;
    border-radius:8px; text-align:center; }
  .stat-num { font-size:24px; font-weight:bold; color:#148F77; }
</style>
</body>

```

Lógica en JavaScript (paso a paso)

```

// Array inicial de productos (objetos con 4 propiedades)
let productos = [
  { nombre:"Auriculares", precio:59.99, categoria:"Electrónica", emoji:"🎧" },
  { nombre:"Mochila",      precio:34.50, categoria:"Accesorios",  emoji:"🎒" },
  { nombre:"Taza Mágica",  precio:12.00, categoria:"Hogar",      emoji:"☕" },
  { nombre:"Teclado RGB",  precio:89.99, categoria:"Electrónica", emoji:"⌨️" },
  { nombre:"Planta Suculenta", precio:8.99, categoria:"Hogar",    emoji:"🌱" },
];

function agregarProducto() {
  const nombre    = document.getElementById("inp-nombre").value;
  const precio    = Number(document.getElementById("inp-precio").value);
  const categoria = document.getElementById("inp-cat").value;
  const emoji     = document.getElementById("inp-emoji").value || "📦";

  // Validar que los campos obligatorios tengan datos
  if (!nombre || !precio || !categoria) {
    alert("Completa nombre, precio y categoría");
    return;
  }

  // Crear el objeto producto y agregarlo al array

```



```

        "    <span class=\"prod-cat\">" + p.categoria + "</span><br>" +
        "    <button class=\"btn-del\" onclick=\"eliminarProducto(" + i + ")\">
Quitar</button>" +
        "</div>";
    }

    calcularResumen();
}

render();

```

⚡ Mini-reto para ti

Agrega un campo "stock: número" al objeto y muestra "⚠ Sin stock" en rojo si el stock es 0.

Agrega un buscador que filtre el catálogo por nombre o categoría mientras el usuario escribe (oninput).

Bonus final: Implementa un "carrito de compras". Agrega un botón " Agregar al carrito" y un panel lateral que muestre los productos seleccionados y el total a pagar.



¡Nivel 2 Completado! — Lo que aprendiste



Arrays

Declarar, acceder por índice [i], medir con .length, agregar con push() y eliminar con splice(). El bloque constructor de cualquier app de datos.



Bucle for

Repetir lógica N veces usando un contador i. Recorrer arrays completos, calcular acumuladores y generar HTML dinámico sin repetir código.



Objetos { }

Representar entidades con múltiples propiedades. Arrays de objetos como mini-bases de datos. Acceder y modificar propiedades con la notación de punto.



DOM Dinámico

Generar HTML complejo desde datos. Pasar índices en eventos onclick para identificar elementos. El patrón "datos → render → interacción" que usa toda app moderna.

¿Qué sigue? — Nivel 3 (próximas semanas)

- ◆ fetch() y APIs REST: traer datos reales de internet (clima, criptos, películas).
- ◆ localStorage: guardar datos en el navegador sin base de datos.
- ◆ Métodos de array modernos: .map(), .filter(), .find(), .reduce().
- ◆ Clases y Programación Orientada a Objetos: organizar código como lo hace la industria.
- ◆ Introducción a un framework: los fundamentos de React o Vue en 1 sesión.

El patrón que aprendiste en 10 ejercicios

PASO 1



Definir los datos

```
let lista = [{...},{...}]
```

PASO 2



Renderizar con for

```
for(i) { html += ... }
```

PASO 3



Evento modifica + re-render

```
fn(idx) { lista[idx]=... }
```